Professional Study Notes

Terraform Part 2

Installation & Configuration

Beginner to Advanced • English + Kannada Explanations

AWS, Jenkins & EKS Practical Examples

Learning Objectives

After completing this part, you should be able to:

- Install Terraform on Windows, Linux, and macOS.
- Understand Terraform versions, provider versions, and compatibility.
- Configure AWS providers and understand provider plugins.
- Understand Terraform configuration files and their purpose.
- Use essential Terraform CLI commands.
- Initialize Terraform projects using terraform init.
- Troubleshoot common installation, provider, and initialization errors.
- Answer scenario-based Terraform interview questions confidently.

> ಕನ್ನಡ ಈ part ನಲ್ಲಿ Terraform ಅನ್ನು install ಮಾಡುವುದು, AWS ಜೊತೆ configure ಮಾಡುವುದು, project files ಅರ್ಥಮಾಡಿಕೊಳ್ಳುವುದು ಮತ್ತು terraform init errors troubleshoot ಮಾಡುವುದು ಕಲಿಯುತ್ತೇವೆ.

Topic 1 — Terraform Installation

1. Definition — ಇದು ಏನು?

Terraform installation is the process of installing the Terraform CLI (Command-Line Interface) on a computer so that we can write, validate, plan, and apply Infrastructure as Code (IaC).

Terraform is developed by HashiCorp and is available for Windows, Linux, and macOS.

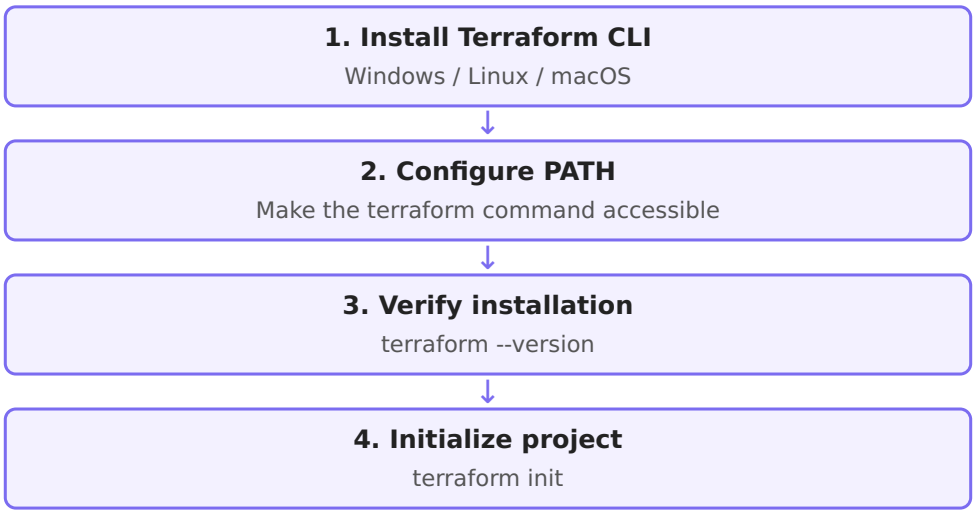
> ಕನ್ನಡ Terraform installation ಅಂದರೆ ನಮ್ಮ system ನಲ್ಲಿ Terraform command ಅನ್ನು ಬಳಸಲು ಅಗತ್ಯವಿರುವ software ಅನ್ನು install ಮಾಡುವುದು.

2. Purpose — ಯಾಕೆ ಬಳಸುತ್ತೇವೆ?

- To create and manage AWS resources using code.
- To automate infrastructure provisioning.
- To integrate infrastructure changes with Jenkins CI/CD.
- To manage infrastructure consistently across development, testing, and production.

For example, instead of manually creating an EC2 instance in the AWS Console, we can define it in Terraform configuration files.

3. Working — ಹೇಗೆ ಕೆಲಸ ಮಾಡುತ್ತದೆ?



ಕನ್ನಡ Install ಮಾಡಿದ ನಂತರ Terraform executable ಇರುವ folder ಅನ್ನು PATH ಗೆ ಸೇರಿಸಿದರೆ, ಯಾವುದೇ terminal ನಿಂದ terraform command execute ಮಾಡಬಹುದು.

4. Practical Example — Installation

A. Windows installation

Official download page: HashiCorp Terraform Installation Guide. You can install Terraform using Windows Package Manager:

```
winget install Hashicorp.Terraform
```

Close and reopen PowerShell or Git Bash, then verify:

```
terraform --version
where.exe terraform
```

Example output:

```
Terraform v1.16.5
on windows_amd64
```

> The version above is an example from your previously described environment; your installed version may differ.

B. Linux installation

On Ubuntu/Debian, use HashiCorp's official installation instructions for the correct repository and package. After installation:

```
terraform --version
which terraform
```

C. macOS installation

Using Homebrew:

```
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
terraform --version
```

D. Configure PATH manually

If Terraform is installed but the terminal says terraform: command not found, add the directory containing the executable to your system PATH.

On Windows, the PATH entry must point to the directory containing terraform.exe, not to the executable itself.

5. Scenario-based Interview Questions & Answers

Q1. Terraform is installed, but PowerShell says the command is not recognized. How do you troubleshoot?

Answer:

1. Run where.exe terraform.
2. Confirm that terraform.exe exists.
3. Verify the PATH environment variable.
4. Open a new terminal after modifying PATH.
5. Run terraform --version again.

> ಕನ್ನಡ File install ಆಗಿದೆಯೇ, PATH ಸರಿಯಾಗಿದೆಯೇ ಎಂದು check ಮಾಡಬೇಕು.

Q2. Can Terraform run without AWS CLI?

Answer: Yes. Terraform's AWS provider can communicate with AWS using supported credential mechanisms, such as environment variables, shared AWS configuration files, IAM roles, or supported workload identity mechanisms. AWS CLI is not mandatory, but it is useful for authentication checks and troubleshooting.

Q3. Terraform works in Git Bash but not in Jenkins. Why?

Answer: Jenkins may run under a different Windows service account with a different PATH, HOME directory, credentials, or permissions. Check the Jenkins service environment and its configured credentials rather than assuming the interactive terminal's environment is shared.

Topic 2 — Terraform Version Compatibility

1. Definition — ಇದು ಏನು?

Version compatibility means ensuring that the Terraform CLI, providers, modules, and required infrastructure features work together correctly.

Terraform projects may specify version requirements for the Terraform CLI and individual providers.

> ಕನ್ನಡ Terraform CLI version ಮತ್ತು AWS provider version project ಗೆ compatible ಆಗಿರಬೇಕು. ಇಲ್ಲದಿದ್ದರೆ initialization, validation ಅಥವಾ resource deployment ಸಮಯದಲ್ಲಿ errors ಬರಬಹುದು.

2. Purpose — ಯಾಕೆ ಬಳಸುತ್ತೇವೆ?

- To prevent unexpected behavior after upgrades.
- To keep local development and Jenkins environments consistent.
- To control which provider versions a project can use.
- To support reliable CI/CD pipelines and production changes.

3. Working — ಹೇಗೆ ಕೆಲಸ ಮಾಡುತ್ತದೆ?

Terraform version requirements are commonly defined in versions.tf.

```
terraform {
  required_version = ">= 1.5.0, < 2.0.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}
```

Understanding the constraints

Constraint	Meaning
>= 1.5.0	Version 1.5.0 or later
< 2.0.0	Exclude Terraform 2.0.0 and later
~> 6.0	AWS provider 6.x, below 7.0.0
= 6.42.0	Exactly version 6.42.0

The ~> operator is called the pessimistic version constraint. The number of version components matters: ~> 6.42.0 allows versions from 6.42.0 up to, but not including, 6.43.0.

> ಕನ್ನಡ Version constraint ಅಂದರೆ ಯಾವ versions project ನಲ್ಲಿ ಬಳಸಲು ಅನುಮತಿ ಇದೆ ಎಂಬ ನಿಯಮ.

Three important version concepts

Concept	Meaning
Terraform CLI version	Runs commands such as init, plan, and apply.

Concept	Meaning
Provider version	Defines the plugin implementation used to manage AWS resources.
Module version	Controls the reusable infrastructure module version when using a versioned module source.

These versions are related but are not the same thing.

4. Practical Example — Your AWS project

Suppose your project uses:

- Terraform CLI: 1.16.5
- AWS provider: 6.42.0
- Backend: Amazon S3
- CI/CD: Jenkins

First, check the CLI:

```
terraform --version
```

Check which provider versions are selected:

```
terraform providers
```

Check the version constraint in versions.tf:

```
terraform {
  required_version = ">= 1.5.0, < 2.0.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}
```

The example constraint permits AWS provider 6.42.0, but it does not force that exact version. The lock file is used to record the selected provider version.

Why is .terraform.lock.hcl important?

The dependency lock file records selected provider versions and checksums. Commit it to Git so your team and Jenkins can reproduce provider selections more reliably.

Do not confuse the lock file with Terraform state:

- .terraform.lock.hcl → provider dependency selections and checksums.
- terraform.tfstate → Terraform's record of managed infrastructure objects and their attributes.

5. Scenario-based Interview Questions & Answers

Q1. My local machine uses AWS provider 6.42.0, but Jenkins installs a different version. What should I do?

Answer:

1. Commit versions.tf and .terraform.lock.hcl.
2. Check that Jenkins checks out the correct branch and commit.
3. Run terraform init in the Jenkins workspace.
4. Avoid terraform init -upgrade in routine deployment jobs unless an upgrade is intentional.
5. Review and test any provider upgrades before merging.

Q2. What is the difference between ~> 6.0 and = 6.42.0?

Answer: ~> 6.0 allows compatible provider releases within the 6.x major version range. = 6.42.0 allows only version 6.42.0. The lock file then records the version selected during initialization.

Q3. Can we upgrade Terraform and its provider in production directly?

Answer: Avoid untested upgrades. Test in a lower environment, review release notes, initialize the updated dependencies, validate the configuration, inspect the plan, and follow your change-management process before production rollout.

Topic 3 — Terraform Providers

1. Definition — ಇದು ಏನು?

A Terraform provider is a plugin that allows Terraform to communicate with an external platform or service and manage its resources. Examples include:

- AWS provider — Amazon Web Services
- AzureRM provider — Microsoft Azure
- Google provider — Google Cloud
- Kubernetes provider — Kubernetes API
- GitHub provider — GitHub resources

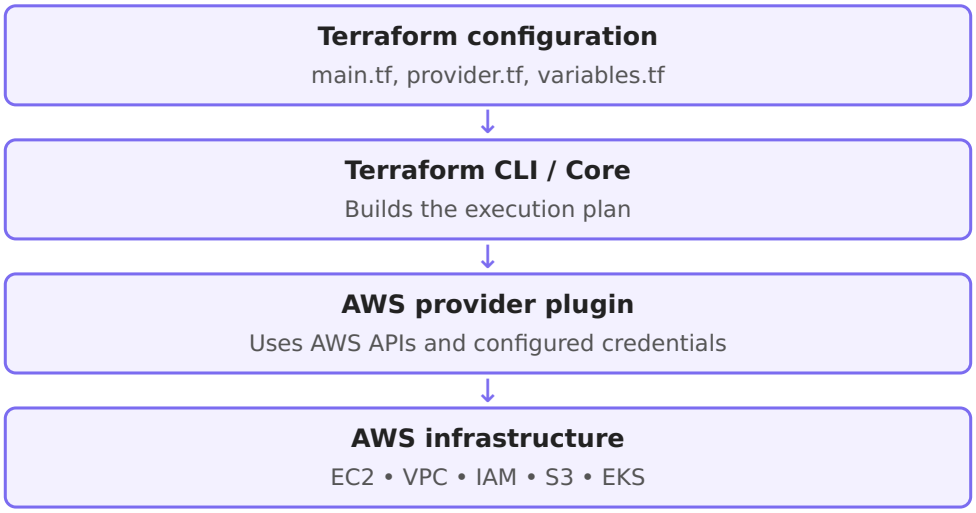
ಕನ್ನಡ Provider ಅಂದರೆ Terraform ಮತ್ತು AWS ಮುಂತಾದ external services ನಡುವೆ communication ಮಾಡಿಸುವ plugin.

2. Purpose — ಯಾಕೆ ಬಳಸುತ್ತೇವೆ?

Terraform core provides the execution engine, but it does not independently contain every platform-specific resource implementation.

Providers allow Terraform to manage infrastructure such as EC2 instances, S3 buckets, VPCs, EKS clusters, and security groups.

3. Working — ಹೇಗೆ ಕೆಲಸ ಮಾಡುತ್ತದೆ?



ಕನ್ನಡ Terraform Core plan ಮಾಡುತ್ತದೆ. AWS provider AWS API ಮೂಲಕ resources create, read, update ಅಥವಾ delete ಮಾಡಲು ಸಹಾಯ ಮಾಡುತ್ತದೆ.

4. Practical Example — Configure the AWS provider

Create a file called provider.tf:

```
provider "aws" {
  region = var.aws_region

  allowed_account_ids = [
    var.aws_account_id
  ]

  default_tags {
    tags = {
      Environment = "dev"
      ManagedBy   = "Terraform"
      Project     = "EKS"
    }
  }
}
```

In variables.tf:

```
variable "aws_region" {
  description = "AWS region for infrastructure"
  type        = string
  default     = "us-east-1"
}

variable "aws_account_id" {
  description = "Expected AWS account ID"
  type        = string
}
```

Set the account ID using a secure or appropriate configuration method. allowed_account_ids helps prevent accidental operations against an unintended AWS account.

Configure credentials separately

For local development, the AWS provider can use a shared credentials profile:

```
aws configure --profile terraform-dev
```

Then select that profile in the shell before running Terraform, or configure a supported profile mechanism.

For Jenkins, prefer securely managed credentials or an IAM role with least-privilege permissions. Do not hardcode access keys in .tf files or commit them to Git.

Validate the identity before deploying:

```
aws sts get-caller-identity
```

> Important: The AWS provider configuration does not install the provider. Provider installation happens during terraform init.

5. Scenario-based Interview Questions & Answers

Q1. Terraform says, “Failed to query available provider packages.” What will you check?

Answer:

- Internet or network access to the provider registry.
- required_providers source and version constraints.
- Proxy, firewall, and registry access.
- Whether .terraform.lock.hcl contains a compatible selection.
- Whether Jenkins can download the required provider binaries.

Q2. Why should we use allowed_account_ids?

Answer: It adds an account-level safety check to the AWS provider configuration, helping prevent Terraform operations against an unintended AWS account.

Q3. Can a single Terraform project use multiple providers?

Answer: Yes. For example, one configuration can use AWS and Kubernetes providers. Provider aliases allow multiple configurations of the same provider, such as two AWS regions or two accounts.

```
provider "aws" {
  region = "us-east-1"
}

provider "aws" {
  alias  = "ohio"
  region = "us-east-2"
}
```

A resource can select the second configuration using: provider = aws.ohio. The resource must still be declared in the configuration; the provider block alone does not create infrastructure.

Topic 4 — Terraform Configuration Files

1. Definition — ಇದು ಏನು?

Terraform configuration files are files written in HashiCorp Configuration Language (HCL) that describe the infrastructure Terraform should manage.

The standard file extension is .tf. Variable definition files commonly use .tfvars or .auto.tfvars.

> ಕನ್ನಡ Terraform configuration files ಅಂದರೆ ಯಾವ AWS resources ಬೇಕು, ಅವುಗಳ configuration ಏನು, ಯಾವ region ನಲ್ಲಿ create ಮಾಡಬೇಕು ಎಂಬುದನ್ನು ಬರೆಯುವ files.

2. Purpose — ಯಾಕೆ ಬಳಸುತ್ತೇವೆ?

- To organize infrastructure code.
- To separate providers, variables, resources, and outputs.
- To make configurations reusable and maintainable.
- To support code review and CI/CD.
- To make infrastructure changes traceable through Git.

3. Working — ಹೇಗೆ ಕೆಲಸ ಮಾಡುತ್ತದೆ?

Terraform loads configuration files with the .tf and .tf.json extensions in the working directory and evaluates them together as one configuration.

The filenames help humans organize the code, but Terraform does not require you to place each block in a particular filename.

4. Practical Example — Common Terraform files

`main.tf`
Main infrastructure resources, such as EC2, VPC, or EKS resources.

ಕನ್ನಡ ಮುಖ್ಯ infrastructure code.

`provider.tf`
Provider configuration, such as AWS region and account restrictions.

ಕನ್ನಡ AWS ಜೊತೆ ಸಂಪರ್ಕಿಸುವ provider settings.

`variables.tf`
Input variable declarations, types, descriptions, and optional defaults.

ಕನ್ನಡ ಬದಲಾಯಿಸಬಹುದಾದ input values.

`terraform.tfvars`
Values assigned to declared input variables.

ಕನ್ನಡ Variables ಗೆ actual values ನೀಡುವ file.

`outputs.tf`
Exposes useful values, such as an EC2 public IP or EKS cluster endpoint.

ಕನ್ನಡ Resource create ಆದ ನಂತರ ಬೇಕಾದ output values.

`versions.tf`
Declares Terraform CLI and provider version requirements.

ಕನ್ನಡ Version compatibility ನಿಯಮಗಳು.

`.terraform.lock.hcl`
Locks provider versions and records checksums.

ಕನ್ನಡ Provider dependency selection ಸ್ಥಿರವಾಗಿರಲು ಸಹಾಯ ಮಾಡುತ್ತದೆ.

Additional important files and folders

File or folder	Purpose
data.tf	Data source declarations
locals.tf	Local expressions and reusable values
modules/	Reusable infrastructure components
backend.tf	Backend configuration for Terraform state
.terraform/	Locally initialized working data and downloaded provider/module components
terraform.tfstate	Local state file, when using the local backend
terraform.tfstate.backup	May contain a previous local state snapshot
terraform.tfplan	A saved plan, if you create one using that filename

The actual state location depends on the configured backend. With an S3 backend, the authoritative state is stored remotely rather than in the workspace's local state file.

Example: main.tf

```
resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = var.instance_type

  tags = {
    Name = "terraform-web-server"
  }
}
```

Example: variables.tf

```
variable "ami_id" {
  description = "AMI ID for the EC2 instance"
  type        = string
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}
```

Example: terraform.tfvars

```
ami_id        = "ami-0123456789abcdef0"
instance_type = "t3.micro"
```

> The AMI ID is illustrative. Use an appropriate AMI for the selected region and operating system.

Example: outputs.tf

```
output "instance_id" {
  description = "Created EC2 instance ID"
  value       = aws_instance.web.id
}
```

Example: versions.tf

```
terraform {
  required_version = ">= 1.5.0, < 2.0.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}
```

5. Scenario-based Interview Questions & Answers

Q1. Is main.tf mandatory?

Answer: No. Terraform loads all supported configuration files in the working directory. You could place the entire configuration in a single file or divide it into several files.

Q2. What happens if terraform.tfvars is missing?

Answer: If required variables have no defaults and no other values are supplied, Terraform prompts for values in an interactive run or reports missing input in a non-interactive run. Jenkins pipelines should supply required variables explicitly.

Q3. Should we commit .terraform/ to Git?

Answer: Normally, no. Add .terraform/ to .gitignore. Commit the dependency lock file, and manage remote state securely.

```
.terraform/
*.tfstate
*.tfstate.*
*.tfplan
crash.log
```

Review any exceptions carefully. Some teams deliberately keep specific plan artifacts outside Git, in secure pipeline storage, rather than committing them.

Q4. Is terraform.tfvars safe for passwords and access keys?

Answer: Not automatically. It can contain sensitive information, but Terraform variables marked sensitive = true only suppress certain displays; they do not guarantee that values are absent from state or plan files. Use a secrets manager or secure CI/CD credential mechanism, and protect state and saved plans.

Topic 5 — Terraform CLI Commands

1. Definition — ಇದು ಏನು?

Terraform CLI is the command-line interface used to interact with Terraform projects.

It allows engineers to initialize a project, format configuration, validate syntax, inspect proposed infrastructure changes, deploy resources, and manage state.

> ಕನ್ನಡ Terraform CLI ಅಂದರೆ terminal ಮೂಲಕ Terraform project ಅನ್ನು control ಮಾಡುವ commands.

2. Purpose — ಯಾಕೆ ಬಳಸುತ್ತೇವೆ?

- To automate infrastructure operations.
- To validate code before deployment.
- To review changes before applying them.
- To troubleshoot configuration and state issues.
- To integrate Terraform into Jenkins pipelines.

3. Working — ಹೇಗೆ ಕೆಲಸ ಮಾಡುತ್ತದೆ?

Terraform CLI commands read configuration files, load providers and modules as needed, and interact with state and remote APIs depending on the command.

Commands do different jobs: validate checks configuration consistency, plan proposes changes, and apply executes the proposed changes.

4. Practical Example — Essential commands

Run these commands from your Terraform project directory.

Command	Purpose
terraform --version	Show Terraform CLI version
terraform -help	Show general command help
terraform init	Initialize the working directory
terraform fmt	Format configuration files
terraform fmt -check	Check formatting without rewriting files
terraform validate	Validate configuration syntax and consistency
terraform providers	Show provider requirements in the configuration
terraform plan	Preview proposed changes
terraform apply	Execute changes after approval
terraform output	Display root module outputs
terraform state list	List objects recorded in state
terraform show	Display state or a saved plan
terraform destroy	Propose and execute resource destruction after confirmation

Recommended command sequence

```
terraform init
terraform fmt -check
terraform validate
terraform plan
terraform apply
```

> Important: terraform validate does not guarantee that AWS credentials work, that AWS resources can be created, or that a plan is safe. It checks the configuration itself.

Save and apply a plan

```
terraform plan -out=tfplan
terraform show tfplan
terraform apply tfplan
```

When you apply a saved plan, Terraform executes the decisions recorded in that plan rather than generating a fresh plan for that command. Saved plans can contain sensitive data, so protect them and regenerate them if the configuration or environment changes.

5. Scenario-based Interview Questions & Answers

Q1. What is the difference between terraform plan and terraform apply?

Answer: plan previews the proposed infrastructure changes. apply executes changes, either from a newly generated plan or from a saved plan supplied to the command.

Q2. Does terraform validate connect to AWS?

Answer: It does not perform a full AWS infrastructure deployment or verify that every AWS API operation will succeed. It checks whether the configuration is syntactically valid and internally consistent, with initialized providers and modules available as needed.

Q3. My terraform fmt -check command fails in Jenkins. What should I do?

Answer: Run terraform fmt locally, review the changes, commit the formatted files, and rerun the pipeline. Use terraform fmt -check in CI to enforce consistent formatting.

Q4. What is the purpose of terraform state list?

Answer: It lists the resource instances currently recorded in the selected Terraform state. It does not list every resource in the AWS account.

Topic 6 — Terraform Initialization (terraform init)

1. Definition — ಇದು ಏನು?

terraform init initializes a Terraform working directory. It prepares the directory for Terraform operations by configuring the backend, installing required providers, and retrieving required modules.

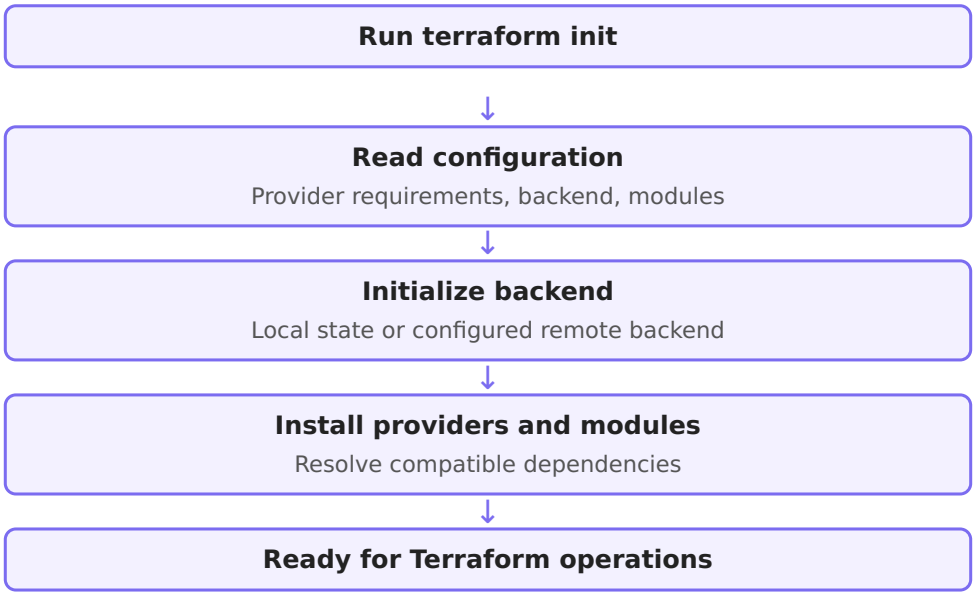
It is generally the first command to run when starting a Terraform project.

ಕನ್ನಡ terraform init ಅಂದರೆ Terraform project ಅನ್ನು ಕೆಲಸ ಮಾಡಲು ready ಮಾಡುವ ಮೊದಲ ಮುಖ್ಯ command. ಇದು ಬೇಕಾದ providers ಮತ್ತು modules download ಮಾಡುತ್ತದೆ ಹಾಗೂ state backend ಅನ್ನು initialize ಮಾಡುತ್ತದೆ.

2. Purpose — ಯಾಕೆ ಬಳಸುತ್ತೇವೆ?

- To initialize the configured state backend.
- To download required provider plugins.
- To download modules used by the configuration.
- To establish provider dependency selections through the lock file.
- To prepare the project for validation, planning, and applying.

3. Working — ಹೇಗೆ ಕೆಲಸ ಮಾಡುತ್ತದೆ?



ಕನ್ನಡ Initialization ಸಮಯದಲ್ಲಿ Terraform backend configuration, provider versions ಮತ್ತು modules ಅನ್ನು prepare ಮಾಡುತ್ತದೆ. terraform init ಸಾಮಾನ್ಯವಾಗಿ AWS infrastructure ಅನ್ನು create ಮಾಡುವುದಿಲ್ಲ.

4. Practical Example — Local AWS project

Navigate to your project:

```
cd /c/terraform-aws-infrastructure
```

Initialize it:

```
terraform init
```

A successful run usually prints a message similar to:

```
Terraform has been successfully initialized!
```

Next, execute:

```
terraform fmt -check
terraform validate
terraform plan
```

Important terraform init options

Command	Purpose
terraform init	Normal initialization
terraform init -upgrade	Upgrade eligible provider and module selections within the configured constraints
terraform init -reconfigure	Reconfigure the backend without migrating existing state
terraform init -migrate-state	Attempt to migrate existing state to the newly configured backend
terraform init -backend=false	Initialize without initializing the backend

Use -upgrade intentionally, because it may select newer dependency versions allowed by the configuration.

Use -reconfigure and -migrate-state carefully. They are not interchangeable: reconfiguration does not migrate existing state, while migration attempts to transfer it.

Practical Example — Your EKS project with S3 backend

Your EKS project uses an S3 backend and a DynamoDB lock table in its existing setup. A simplified backend.tf might look like this:

```
terraform {
  backend "s3" {
    bucket         = "s3buckk8sproject1"
    key            = "eks/terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "terraform-state-lock"
    encrypt        = true
  }
}
```

The bucket, region, state key, and locking configuration must match your actual AWS setup. This example reflects the earlier project configuration; verify it before using it.

Initialize:

```
terraform init
```

If Terraform reports that the backend configuration has changed, first determine whether you are intentionally changing backend settings or migrating existing state. Do not blindly run initialization flags to suppress the error.

Advanced note: DynamoDB locking

The S3 backend's dynamodb_table locking option is deprecated in newer Terraform versions; S3-native lock files can be configured with use_lockfile = true where supported. For example, a newer configuration may use:

```
terraform {
  backend "s3" {
    bucket       = "s3buckk8sproject1"
    key          = "eks/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    use_lockfile = true
  }
}
```

Before removing an existing DynamoDB lock table, coordinate with your team and migrate the locking setup carefully. Existing workflows must use a consistent locking strategy.

5. Scenario-based Interview Questions & Answers

Q1. Why is terraform init required?

Answer: It prepares the working directory, installs the required providers and modules, and initializes the configured backend so Terraform can work with dependencies and state.

Q2. What is the difference between terraform init and terraform init -upgrade?

Answer: Normal initialization respects the existing dependency lock selections where possible. -upgrade considers newer eligible versions and updates selections and lock information accordingly.

Q3. Terraform initialization fails with an S3 AccessDenied error. How do you troubleshoot?

Answer:

1. Check which AWS identity is being used.
2. Verify permissions to access the specified S3 bucket and state object.
3. Check the backend region, bucket name, and state key.
4. Check permissions required for state locking if locking is configured.
5. Check bucket policies, encryption settings, and KMS permissions where applicable.
6. Retry initialization after correcting the underlying issue.

Q4. Terraform says, “Missing key LockID” while using DynamoDB locking. What could be wrong?

Answer: The configured DynamoDB lock table may not have the expected partition key schema. The legacy S3 backend DynamoDB locking configuration expects a string partition key named LockID. Verify the table schema, backend settings, and Terraform version before changing the table or migrating the locking mechanism.

Q5. Will terraform init delete my EC2 instance?

Answer: Normally, no. Initialization prepares dependencies and backend configuration; it does not ordinarily destroy managed resources. However, backend migration and state reconfiguration are sensitive operations that must be performed carefully.

Topic 7 — Terraform Environment Configuration and Best Practices

1. Definition — ಇದು ಏನು?

Environment configuration is the process of supplying Terraform with the correct region, credentials, variables, backend settings, and execution environment for a particular deployment.

> ಕನ್ನಡ Development, testing, production ಮುಂತಾದ environments ನಲ್ಲಿ Terraform ಸರಿಯಾಗಿ ಕೆಲಸ ಮಾಡಲು ಅಗತ್ಯವಿರುವ settings configure ಮಾಡುವುದು.

2. Purpose — ಯಾಕೆ ಬಳಸುತ್ತೇವೆ?

- To deploy resources into the correct AWS account and region.
- To avoid exposing credentials in source code.
- To maintain separate development and production settings.
- To enable consistent Jenkins CI/CD execution.
- To prevent accidental infrastructure changes in the wrong environment.

3. Working — ಹೇಗೆ ಕೆಲಸ ಮಾಡುತ್ತದೆ?

Terraform can obtain configuration values from several sources, including:

- Configuration files and variable files.
- Command-line options such as -var and -var-file.
- Environment variables such as TF_VAR_aws_region.
- AWS credential profiles, environment credentials, and IAM roles.
- Backend configuration supplied in supported configuration or initialization options.

For Terraform input variables, explicit command-line variable values take precedence over environment variables. Terraform also has defined precedence rules for variable files and defaults.

4. Practical Example — AWS + Jenkins

Step 1: Configure AWS credentials locally

```
aws configure --profile terraform-dev
```

Verify the selected AWS identity:

```
aws sts get-caller-identity --profile terraform-dev
```

Step 2: Define your Terraform variables

```
variable "aws_region" {
  type    = string
  default = "us-east-1"
}

variable "instance_type" {
  type    = string
  default = "t3.micro"
}
```

Step 3: Use environment variables if needed

In Git Bash:

```
export TF_VAR_aws_region="us-east-1"
```

Terraform can automatically use this value for the declared aws_region input variable.

Step 4: Execute Terraform

```
terraform init
terraform validate
terraform plan
```

Step 5: Use secure credentials in Jenkins

For your Jenkins pipeline, use a properly configured Jenkins credential or an IAM role where supported. Ensure the pipeline uses the expected region, AWS account, Terraform working directory, and provider lock file.

Do not store AWS access keys or secret keys directly in the Jenkinsfile.

5. Scenario-based Interview Questions & Answers

Q1. Terraform works locally but fails in Jenkins with AWS credential errors. Why?

Answer: Jenkins may run under a different service account, with different environment variables, profiles, permissions, or credentials. Check the identity using aws sts get-caller-identity in the Jenkins execution context and verify the credentials binding or IAM role configuration.

Q2. How do you manage different AWS environments?

Answer: Use separate variable files, appropriately scoped credentials, separate state locations, and environment-specific configurations. For example:

```
terraform plan -var-file="dev.tfvars"
```

For production, add approval gates and stronger access controls.

Q3. What is the difference between a variable and a local value?

Answer: An input variable allows a caller to supply a value. A local value defines a reusable expression inside the configuration.

```
variable "project_name" {
  type    = string
  default = "eks-project"
}

locals {
  resource_name = "${var.project_name}-cluster"
}
```

Topic 8 — Common Terraform Installation and Initialization Errors

These scenarios are especially useful for DevOps and infrastructure support interviews.

Error / symptom	Likely cause	Recommended action
terraform: command not found	Incorrect PATH or missing installation	Verify the executable and PATH
Unsupported Terraform Core version	CLI version violates the configured constraint	Install a compatible CLI version or deliberately update the constraint
Failed to query available provider packages	Registry, network, or provider configuration problem	Check connectivity and required_providers
Inconsistent dependency lock file	Dependency selections do not match the current configuration	Review version constraints and reinitialize appropriately
No valid credential sources found	AWS credentials are unavailable	Check identity, profiles, environment, or IAM role
AccessDenied on S3 backend	Missing state or bucket permissions	Review IAM, bucket policy, encryption and KMS access
Backend configuration changed	Backend settings differ from previous initialization	Determine whether to reconfigure or migrate state
Missing key LockID	Incorrect DynamoDB lock-table schema	Verify the legacy locking configuration
InvalidClientTokenId	Invalid, expired, or incorrect AWS credentials	Refresh credentials and verify the selected AWS identity
Unsupported argument	Configuration argument is invalid for the selected provider version	Check provider documentation and the locked version

ಕನ್ನಡ Interview tip: Error message ಅನ್ನು ಮೊದಲು ಸಂಪೂರ್ಣವಾಗಿ ಓದಿ. ನಂತರ configuration, version, credentials, permissions, backend ಮತ್ತು network ಅನ್ನು ಕ್ರಮವಾಗಿ ಪರಿಶೀಲಿಸಿ. Error ಕಾಣುತ್ತಿದ್ದಂತೆ state delete ಮಾಡಬೇಡಿ.

Topic 9 — Complete Practical Lab

This lab combines installation, version compatibility, provider configuration, Terraform files, CLI commands, and initialization.

Step 1: Create a project directory

```
mkdir terraform-part2-lab
cd terraform-part2-lab
```

Step 2: Create versions.tf

```
terraform {
  required_version = ">= 1.5.0, < 2.0.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}
```

Step 3: Create provider.tf

```
provider "aws" {
  region = var.aws_region
}
```

Step 4: Create variables.tf

```
variable "aws_region" {
  description = "AWS deployment region"
  type        = string
  default     = "us-east-1"
}

variable "ami_id" {
  description = "AMI ID available in the selected region"
  type        = string
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}
```

Step 5: Create main.tf

```
resource "aws_instance" "demo" {
  ami           = var.ami_id
  instance_type = var.instance_type

  tags = {
    Name        = "terraform-part2-demo"
    ManagedBy   = "Terraform"
    Environment = "dev"
  }
}
```

Step 6: Create outputs.tf

```
output "instance_id" {
  description = "ID of the demo EC2 instance"
  value       = aws_instance.demo.id
}

output "public_ip" {
  description = "Public IP of the demo EC2 instance"
  value       = aws_instance.demo.public_ip
}
```

Step 7: Initialize and validate

```
terraform init
terraform fmt
terraform validate
terraform providers
```

Step 8: Supply an appropriate AMI

Create terraform.tfvars:

```
aws_region    = "us-east-1"
ami_id        = "ami-0123456789abcdef0"
instance_type = "t3.micro"
```

Replace the illustrative AMI with a valid, suitable AMI for your chosen region and operating system.

Step 9: Preview the changes

```
terraform plan
```

If you intend to create the EC2 instance, confirm the account, region, AMI, network configuration, security requirements, and cost implications before running:

```
terraform apply
```

> This lab is intended for a controlled test environment. Creating an EC2 instance can incur AWS charges. If you later want to remove the lab resources, review the destruction plan carefully before running terraform destroy.

Topic 10 — Advanced Interview Preparation

Here are additional questions that test practical understanding beyond command memorization.

Q1. What is the difference between terraform init and terraform validate?

Answer: Initialization prepares backend and dependencies. Validation checks the consistency and syntax of the Terraform configuration.

Q2. Why should .terraform.lock.hcl be committed to Git?

Answer: It records selected provider versions and checksums so that different environments can reproduce provider selections more reliably.

Q3. Does terraform init -upgrade upgrade Terraform CLI itself?

Answer: No. It upgrades eligible provider and module selections within their constraints. The CLI must be upgraded separately.

Q4. What happens if two files declare the same Terraform resource address?

Answer: Terraform treats all files in the directory as one configuration. Duplicate resource declarations with the same type and name in the same module generally cause a duplicate declaration error.

Q5. Can you use one provider configuration for multiple AWS regions?

Answer: Yes, by declaring multiple provider configurations and using aliases where needed. Resources or modules can explicitly select the intended configuration.

Q6. What is the difference between backend configuration and provider configuration?

Answer: The backend determines where Terraform stores and coordinates state. The provider communicates with the infrastructure platform to manage resources. An S3 backend and AWS provider can both interact with AWS, but they serve different purposes.

Q7. Why might initialization succeed locally but fail in Jenkins?

Answer: Differences in network access, environment variables, credentials, workspace files, provider lock files, CLI versions, or backend permissions can cause different outcomes.

Q8. What is the safest way to upgrade an AWS provider in a production project?

Answer: Review the release notes, update the constraint if required, run terraform init -upgrade in a controlled branch, validate and review the plan, test in a lower environment, and promote the tested change through the approved deployment process.

Quick Revision Sheet

Topic	Must-remember point
Terraform installation	Install CLI and verify PATH
Version compatibility	CLI, provider, and module versions are different
Provider	Plugin that communicates with an external platform
Configuration files	Organize infrastructure definitions
versions.tf	Declares CLI and provider requirements
.terraform.lock.hcl	Records selected provider versions and checksums
terraform init	Initializes backend and dependencies
terraform fmt	Formats Terraform configuration
terraform validate	Checks configuration consistency
terraform plan	Previews infrastructure changes
terraform apply	Executes infrastructure changes
terraform.tfvars	Supplies input variable values
AWS credentials	Use secure credential mechanisms
Jenkins	Keep versions, credentials, and execution environment consistent

Self-Assessment Quiz

1. Which command initializes a Terraform working directory?

A) terraform plan
B) terraform init
C) terraform apply

2. Which file records selected provider versions and checksums?

A) outputs.tf
B) .terraform.lock.hcl
C) terraform.tfstate

3. Which configuration block declares the AWS provider version requirement?

A) required_providers
B) output
C) locals

4. What should you do before applying a production plan?

A) Delete the state file
B) Review the plan and follow approval controls
C) Always run init -upgrade

Answer key: 1 → B (terraform init) • 2 → B (.terraform.lock.hcl) • 3 → A (required_providers) • 4 → B (Review the plan and follow approval controls)

Study recommendation: Practice these topics in your existing AWS Terraform and EKS projects. For interviews, be ready to explain not only what each command does, but also how you would troubleshoot a failure in a Jenkins pipeline.